CENTRO DE ENSINO SUPERIOR DE MARINGÁ – UNICESUMAR
CURSO DE ENGENHARIA DE SOFTWARE
DISCIPLINA: ENGENHARIA DE SOFTWARE

Arthur Starke Rodrigues
Murilo Ferreira Wolski

ANÁLISE E CORREÇÃO DE ERROS COMUNS EM DIAGRAMAS DE CLASSES DA UML: APLICAÇÃO EM UM SISTEMA DE RESTAURANTE, CARDÁPIO E PEDIDOS

TEMA: DIAGRAMA DE CLASSES - ERROS COMUNS

MARINGÁ
2026

SUMÁRIO

1 INTRODUÇÃO 3

2 FUNDAMENTAÇÃO TEÓRICA 4

3 MODELAGEM DO CENÁRIO E ANÁLISE DE ERROS COMUNS 5

3.1 Erro 1: Atributo que deveria ser Classe 5

3.2 Erro 2: Multiplicidade Omitida 6

3.3 Erro 3: Herança no Lugar de Associação 7

3.4 Erro 4: Composição Indevida 8

3.5 Erro 5: Ausência de Classe de Associação / Pagamento 9

4 ANÁLISE E DECISÕES DE MODELAGEM 10

5 CONCLUSÃO 11

REFERÊNCIAS 12

1 INTRODUÇÃO

O objetivo deste trabalho é estudar, analisar e exemplificar visual e teoricamente os erros mais frequentes cometidos durante a construção de Diagramas de Classes utilizando a Linguagem de Modelagem Unificada (UML). A modelagem orientada a objetos desempenha um papel crítico no ciclo de vida do desenvolvimento de software, atuando como ponte entre o levantamento de requisitos e a implementação do código-fonte.

Para ilustrar de maneira prática e contextualizada os conceitos abordados, adotou-se como cenário de software um Sistema de Gestão de Restaurante, Cardápio e Pedidos. Este domínio abrange funcionalidades do cotidiano do estabelecimento, como o gerenciamento de itens do cardápio, registro de pedidos por garçons, controle de mesas e emissão de pagamentos.

Neste documento, são diagnosticados cinco erros típicos de modelagem estática: uso de atributos simples para conceitos complexos, omissão de multiplicidades, herança inadequada no lugar de associação, aplicação incorreta de composição forte e falta de suporte para histórico de pagamentos. Para cada caso, apresenta-se o diagrama incorreto, a explicação do vício conceitual e o diagrama corrigido com a notação UML adequada.

2 FUNDAMENTAÇÃO TEÓRICA

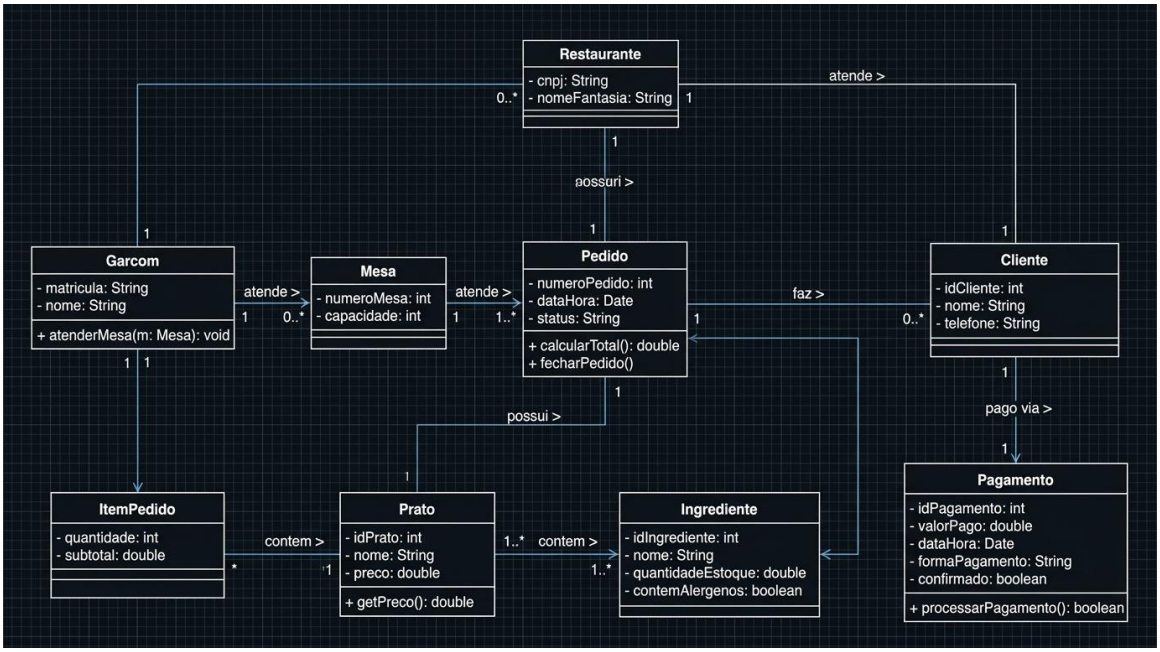
O Diagrama de Classes é o principal artefato estático da UML, responsável por descrever a estrutura de um sistema através do mapeamento de suas classes, atributos, métodos, modificadores de visibilidade e os relacionamentos existentes entre os objetos.

Uma modelagem precisa exige o rigoroso respeito à sintaxe e semântica da notação UML. Pequenos deslizes conceituais durante a elaboração do diagrama podem propagar falhas estruturais graves para o código-fonte, gerando classes com responsabilidades excessivas, alto acoplamento, dependências circulares e impossibilidade de reuso de código. O domínio sobre a diferenciação de relacionamentos — como associação simples, agregação fraca, composição forte, dependência e herança — é fundamental para garantir a estabilidade e a clareza da arquitetura do sistema.

3 MODELAGEM DO CENÁRIO E ANÁLISE DE ERROS COMUNS

A seguir, apresentam-se os cinco erros mais comuns identificados na modelagem do cenário de restaurante, juntamente com os códigos para geração dos diagramas em ferramentas CASE (como PlantUML ou Mermaid) e a fundamentação da correção.

Figura 1 - Diagrama de Classes Completo Corrigido



Fonte: elaborado pelos autores (2026).

3.1 Erro 1: Atributo que deveria ser Classe

Descrição do Erro:

Consiste em tratar um conceito estruturado do domínio — que possui identificação, atributos próprios e ciclo de vida independente — como um mero atributo do tipo primitivo (texto ou número) dentro de outra classe.

Cenário Incorreto:

Na classe Prato, o ingrediente principal foi modelado apenas como um atributo ingredientes: String. Essa abordagem impede que o sistema gerencie o estoque dos ingredientes, registre alérgenos ou associe múltiplos pratos ao mesmo ingrediente.

Explicação da Correção:

A criação da classe Ingrediente permite controlar o estoque e associar cada ingrediente a múltiplos pratos via uma associação N:M (muitos para muitos), garantindo a reutilização e o controle preciso do domínio.

3.2 Erro 2: Multiplicidade Omitida

Descrição do Erro:

Omitir a cardinalidade nas extremidades das linhas de associação. A ausência de multiplicidade gera ambiguidade, impedindo que os desenvolvedores saibam se uma relação permite nenhum, um ou muitos elementos.

Cenário Incorreto:

Entre a classe Pedido e a classe ItemPedido, foi desenhada uma linha simples sem indicação numérica de cardinalidade.

Explicação da Correção:

A inclusão explícita das multiplicidades determina rigorosamente que um Pedido deve conter obrigatoriamente um ou mais (1..*) itens de pedido, enquanto um ItemPedido pertence a exatamente um (1) pedido específico.

3.3 Erro 3: Herança no Lugar de Associação

Descrição do Erro:

Empregar o relacionamento de herança/generalização (representado pelo triângulo vazado) para unir duas classes que mantêm apenas um relacionamento funcional de uso, violando o princípio fundamental "É-Um" (Is-A).

Cenário Incorreto:

A classe Garcom foi desenhada herdando diretamente de Mesa, expressando erroneamente que o garçom "é uma" mesa no sistema.

Explicação da Correção:

A herança foi substituída por uma associação simples com multiplicidade 1 para 0..*, indicando que um garçom é responsável por atender zero ou várias mesas, sem alterar a hierarquia de tipos dos objetos.

3.4 Erro 4: Composição Indevida

Descrição do Erro:

Utilizar o símbolo de composição forte (losango preto preenchido) em associações onde as partes possuem ciclo de vida independente em relação ao objeto "todo". Na composição, se o elemento pai for destruído, todas as partes associadas são compulsoriamente eliminadas do sistema.

Cenário Incorreto:

Modelação de uma composição forte entre a classe Restaurante e a classe Cliente, sugerindo que o cancelamento ou fechamento do restaurante resultaria na exclusão física dos cadastros dos clientes.

Explicação da Correção:

A relação foi corrigida para uma associação simples. O cliente possui um ciclo de vida autônomo e seu cadastro deve permanecer preservado no banco de dados, independentemente do status operacioanal do restaurante.

3.5 Erro 5: Ausência de Classe de Associação / Pagamento

Descrição do Erro:

Tratar operações transacionais de valor financeiro apenas como métodos temporários soltos ou atalhos de dependência, sem encapsular o histórico e as propriedades do pagamento em uma entidade própria.

Cenário Incorreto:

O pagamento de um pedido era efetuado invocando diretamente um método fecharPedido() que recebia um texto simples formaPagamento: String, sem gerar um registro histórico do pagamento.

Explicação da Correção:

A criação da classe Pagamento devidamente associada ao Pedido permite armazenar os metadados da transação financeira (data, hora, forma de pagamento e status da confirmação), garantindo a auditabilidade e integridade fiscal do sistema.

4 ANÁLISE E DECISÕES DE MODELAGEM

A realização deste estudo prático demonstrou o impacto direto das decisões de modelagem na qualidade final de um software. A principal dificuldade encontrada pela dupla residiu na escolha correta do tipo de relacionamento (associação simples vs. agregação vs. composição) e no detalhamento das cardinalidades reais sem sobrecarregar o diagrama com excesso de acoplamento.

Ao isolar e corrigir os cinco erros comuns apresentados, garantiu-se que o modelo de domínio do restaurante respeitasse as regras de negócio reais, mantendo as classes coesas, reutilizáveis e totalmente prontas para o mapeamento e implementação no código-fonte.

5 CONCLUSÃO

Este trabalho atingiu plenamente seus objetivos ao analisar, diagnosticar e corrigir os erros mais recorrentes na elaboração de Diagramas de Classes da UML, utilizando como base um cenário funcional de restaurante. A identificação de equívocos de notação — como heranças inadequadas, falta de cardinalidade e mau uso de composições — permitiu consolidar o aprendizado prático e teórico exigido na disciplina de Engenharia de Software.

As decisões tomadas reforçam a relevância da utilização de ferramentas CASE e do cumprimento estrito das padronizações da UML e da ABNT para a documentação eficiente de sistemas.

REFERÊNCIAS

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2019.